Chhaap: Warli LoRA training on a free Colab T4

1. Runtime > Change runtime type > T4 GPU.
2. `warli_train.zip` (45 cleaned 512x512 Warli images) is already in the project folder. To rebuild it: `cd data/clean/warli && zip -r ../../../warli_train.zip train && cd -`
3. Run the cells below in order. The upload cell asks for `warli_train.zip`.
4. The last cell downloads `warli_lora_outputs.zip`. Unzip it into the project's `models/` folder. It holds `warli_lora.safetensors`, `checkpoints/warli_lora_{500,1000,1500}.safetensors` and `train_loss.csv`. The app loads the LoRA automatically on its next start.

Captions are made here with `Salesforce/blip-image-captioning-base` (the laptop's connection is too slow for the 1 GB model) and every caption starts with the trigger word `warli_style`.

```
!nvidia-smi --query-gpu=name,memory.total --format=csv
!git clone --depth 1 https://github.com/huggingface/diffusers
!pip -q install ./diffusers
!pip -q install -r diffusers/examples/text_to_image/requirements.txt bitsandbytes tensorboard
```

```
name, memory.total [MiB]
Tesla T4, 15360 MiB
Cloning into 'diffusers'...
remote: Enumerating objects: 3293, done.
remote: Counting objects: 100% (3293/3293), done.
remote: Compressing objects: 100% (2465/2465), done.
remote: Total 3293 (delta 1174), reused 1399 (delta 790), pack-reused 0 (from 0)
Receiving objects: 100% (3293/3293), 10.63 MiB | 14.15 MiB/s, done.
Resolving deltas: 100% (1174/1174), done.
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
  Building wheel for diffusers (pyproject.toml) ... done
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 43.1/43.1 MB 14.2 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.8/44.8 kB 3.6 MB/s eta 0:00:00
```

```
from google.colab import files
up = files.upload()  # choose warli_train.zip
!rm -rf data && mkdir data && unzip -q warli_train.zip -d data && ls data/train | wc -l
```

```
Choose Files warli_train.zip
warli_train.zip(application/zip) - 22003028 bytes, last modified: 10/22/2026 - 100% done
Saving warli_train.zip to warli_train.zip
45
```

```
import json, torch
from pathlib import Path
from PIL import Image
from transformers import BlipProcessor, BlipForConditionalGeneration
proc = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
blip = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base", torch_dtype=to
rows = []
for p in sorted(Path("data/train").glob("*.png")):
    x = proc(images=Image.open(p).convert("RGB"), return_tensors="pt").to("cuda", torch.float16)
    cap = proc.decode(blip.generate(**x, max_new_tokens=30)[0], skip_special_tokens=True)
    rows.append({"file_name": p.name, "text": f"warli_style, {cap}"})
Path("data/train/metadata.jsonl").write_text("\n".join(json.dumps(r) for r in rows))
print(len(rows), rows[:3])
del blip; torch.cuda.empty_cache()
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please s
```

```
preprocessor_config.json: 100%  287/287 [00:00<00:00, 30.0kB/s]
config.json: 100%  4.56k/4.56k [00:00<00:00, 405kB/s]
tokenizer_config.json: 100%  506/506 [00:00<00:00, 46.4kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 12.0MB/s]
tokenizer.json: 100%  711k/711k [00:00<00:00, 34.0MB/s]
special_tokens_map.json: 100%  125/125 [00:00<00:00, 12.8kB/s]
pytorch_model.bin: reconstructing file: 100%  990MB / 990MB, 80.7MB/s
pytorch_model.bin: downloading bytes:  933MB, 74.1MB/s
Loading weights: 100%  473/473 [00:00<00:00, 1364.61it/s]
model.safetensors: reconstructing file: 100%  990MB / 990MB, 71.9MB/s
model.safetensors: downloading bytes:  934MB, 73.5MB/s
```

```
45 [{'file_name': '1620463880505_WAR0507_1313__96564.1687167413.png', 'text': 'warli_style, a red and white
```

```
!accelerate launch --mixed_precision=fp16 diffusers/examples/text_to_image/train_text_to_image_lora.py \
  --pretrained_model_name_or_path=stable-diffusion-v1-5/stable-diffusion-v1-5 \
  --train_data_dir=data/train --caption_column=text --resolution=512 --random_flip \
  --train_batch_size=1 --gradient_accumulation_steps=1 --max_train_steps=1500 \
  --learning_rate=1e-4 --lr_scheduler=constant --lr_warmup_steps=0 --rank=16 \
  --mixed_precision=fp16 --use_8bit_adam --checkpointing_steps=500 --seed=0 \
  --output_dir=lora_out --report_to=tensorboard --logging_dir=logs 2>&1 | tee train_log.txt | grep -v it/s |
```

```
    if not is_torchao_available():
           ~~~~~~~~~~~~~~~~~~~~^^
  File "/usr/local/lib/python3.13/dist-packages/peft/import_utils.py", line 162, in is_torchao_available
    raise ImportError(
    ...<2 lines>...
    )
ImportError: Found an incompatible version of torchao. Found version 0.10.0, but only versions above 0.16.0 a
Traceback (most recent call last):
  File "/usr/local/bin/accelerate", line 10, in <module>
    sys.exit(main())
             ~~~~^^
  File "/usr/local/lib/python3.13/dist-packages/accelerate/commands/accelerate_cli.py", line 50, in main
    args.func(args)
    ~~~~~~~~~^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/accelerate/commands/launch.py", line 1413, in launch_command
    simple_launcher(args)
    ~~~~~~~~~~~~~~~^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/accelerate/commands/launch.py", line 999, in simple_launcher
    raise subprocess.CalledProcessError(returncode=process.returncode, cmd=cmd)
subprocess.CalledProcessError: Command '['/usr/bin/python3', 'diffusers/examples/text_to_image/train_text_to_
```

```
import csv, glob, shutil, os
from tensorboard.backend.event_processing.event_accumulator import EventAccumulator
os.makedirs("out/checkpoints", exist_ok=True)
ev = EventAccumulator(glob.glob("lora_out/logs/**/events.out.tfevents.*", recursive=True)[0]); ev.Reload()
tag = [t for t in ev.Tags()["scalars"] if "loss" in t][0]
with open("out/train_loss.csv", "w", newline="") as f:
    w = csv.writer(f); w.writerow(["step", "loss"]); w.writerows((e.step, e.value) for e in ev.Scalars(tag))
for s in (500, 1000, 1500):
    src = f"lora_out/checkpoint-{s}/pytorch_lora_weights.safetensors"
    if os.path.exists(src): shutil.copy(src, f"out/checkpoints/warli_lora_{s}.safetensors")
shutil.copy("lora_out/pytorch_lora_weights.safetensors", "out/warli_lora.safetensors")
shutil.copy("data/train/metadata.jsonl", "out/captions.jsonl")
shutil.copy("train_log.txt", "out/train_log.txt")
!cd out && zip -qr ../warli_lora_outputs.zip . && ls -R
files.download("warli_lora_outputs.zip")
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
/tmp/ipykernel_653/1162278911.py in <cell line: 0>()
      2 from tensorboard.backend.event_processing.event_accumulator import EventAccumulator
      3 os.makedirs("out/checkpoints", exist_ok=True)
----> 4 ev = EventAccumulator(glob.glob("lora_out/logs/**/events.out.tfevents.*", recursive=True)[0]);
ev.Reload()
      5 tag = [t for t in ev.Tags()["scalars"] if "loss" in t][0]
      6 with open("out/train_loss.csv", "w", newline="") as f:

IndexError: list index out of range
```

Next steps: Explain error